

SGLang-Omni 每日 PR 学习日报

仓库 [sgl-project/sglang-omni](#) · 过去 24 小时已合并 PR · 聚焦 Omni / 多模态 / 多阶段架构

报告日期：2026-06-05 (统计窗口：2026-06-04 ~ 2026-06-05)

14

24h 内合并 PR 总数

5

含真实代码改动 (精讲)

9

文档 / 基建类 (速览)

4

直接触及多阶段/多模态

当日主题分类

性能优化 · Higgs 异步解码 (#692)

注意力后端 · LLaDA2-Uni FlashInfer (#691)

调度器适配 · DLLM Scheduler (#670)

多模态音频输出 · PyAV 编码 (#686)

基准/CI · ASR 并发扫描 (#647)

文档 & 基建 · ×9

本报告遵循"讲透优先于覆盖广"：对 5 个含代码的 PR 逐块拆解并补全背景知识；9 个文档/基建类 PR 在文末以表格速览。每张卡片可点击展开/折叠。

PR #692

性能优化

Higgs TTS

多阶段

[Higgs] Optimize Higgs async decode resolve output handling

作者 @Ratish1 · 状态 merged · 查看 PR ↗

一句话：在 Higgs TTS 的"异步解码"路径里，把只用于输出处理的那块小张量从 GPU 上构建改为直接在 CPU 上构建，省掉一次无谓的 GPU 操作与隐式同步，端到端吞吐 +13~15%。

2. 它解决了什么问题

Higgs 是一个 多码本 (multi-codebook) 自回归 TTS 模型：它每"走一步"不是吐出 1 个 token，而是同时吐出多个 codebook 的离散音频码。为了把 GPU 算得快、又不让 CPU 拖后腿，SGLang-Omni 给它实现了一条 异步解码 (async decode) 流水线——GPU 先把这一步算完并"抢跑"下一步，CPU 在后面慢一拍地把结果搬回主机做整理。

痛点在于：异步路径里那一步"滞后的整理" (resolve) 其实只需要一个 CPU 上的小列表（拿来做输出处理、马上要 `.tolist()` 转成 Python list）。但旧代码不分场景，无论同步还是异步，都把这个结果张量建在 GPU 上。在异步整理阶段这就等于：先在 GPU 上 new 一个张量、再立刻拷回 CPU——做了一趟完全没必要的来回，还可能引入隐式的设备同步，打断 GPU 的"抢跑"节奏。

类比：厨房里大厨 (GPU) 已经把下一道菜的料备好抢先开火了；传菜员 (CPU) 只需要把上一道菜的"小票"念给收银台听。旧做法却让传菜员每次都先跑回厨房、在灶台上重抄一张小票再拿出来念——明明手里那张纸条 (CPU 张量) 就够用。这个 PR 就是让传菜员直接用手里的纸条。

3. 具体做了什么改动

核心文件只有一个：`sglang_omni/models/higgs_tts/model_runner.py`。改动的本质是给内部函数 `_decode_collect_host` 增加一个 keyword-only 参数 `next_token_device`，用它来决定"输出张量建在哪个设备上"。

(a) 给 collect 函数加上"目标设备"开关

```
def _decode_collect_host(

-   self, combined_cpu, result, requests

+   self, combined_cpu, result, requests, *, next_token_device: torch.device | None

) -> None:

    ...

-   result.next_token_ids = torch.tensor(

-       cb0_per_row, dtype=torch.long,

-       device=result.logits_output.next_token_logits.device, # 永远在 GPU 上建

-   )

+   if next_token_device is None:           # 异步 resolve : 直接建 CPU 张量

+       result.next_token_ids = torch.tensor(cb0_per_row, dtype=torch.long)

+   else:                                   # 同步路径 : 建在 logits 所在的 GPU

+       result.next_token_ids = torch.tensor(

+           cb0_per_row, dtype=torch.long, device=next_token_device,

+       )
```

`cb0_per_row` 是一个普通的 Python `list[int]` ——每个请求"第 0 号 codebook"这一步采到的 token。把它喂给 `torch.tensor(...)` 时，不指定 device 就默认建在 CPU，指定了才会触发一次 host→device 拷贝。

(b) 两个调用点分别传入不同的设备

```
# 同步 / CUDA-Graph 路径 (_collect_step_outputs_cg) : 结果要喂给"下一步", 必须留在 GPU

+   next_token_device=result.logits_output.next_token_logits.device,

# 异步 resolve 路径 (post_decode_resolve) : 只做输出处理, 传 None → 建在 CPU

+   next_token_device=None,
```

(c) 为什么异步路径"丢掉 GPU 张量"是安全的？

关键在于异步路径的另一半（`post_decode_launch`，GPU 半边）早就已经把下一步要用的 codebook-0 token 在 GPU 上发布好了。下面是同仓库里 launch 阶段的真实注释（`model_runner.py` 第 84–94 行）：

```
# Set next_token_ids (cb0) from GPU state now, with NO host sync, so the
# AR input chain (next step's input_ids = this step's output_ids) is
# available at launch — the host collect (post_decode_resolve) lags by one step.
result.next_token_ids = (
    self.model._cg_codes_BN[:n_real, 0].clamp_min(0).to(torch.long).clone()
)
```

也就是说：在异步模式下，“驱动下一步自回归”的那份 GPU 张量已经在 launch 阶段产出了；滞后一步的 resolve 只负责“把这一步的结果整理成给用户看的输出”，它要的只是一个 CPU 列表。所以把它建在 CPU 上不仅不影响正确性，反而省掉了“建在 GPU 再拷回来”的浪费。PR 描述也明确：采样数学、logits、温度/top-p/top-k、停止条件全部未动，只动了“输出张量落在哪个设备”。

4. 涉及的知识（重点·小白展开）

► 多码本 TTS / RVQ codebook（多个“声音通道”一起预测）

它是什么：普通语言模型一步预测 1 个文字 token；而神经音频编解码器（如 RVQ，残差向量量化）会把一帧声音压成 **多个并行的离散码**（codebook 0、1、2...）。Higgs 这类模型一步要同时预测好几个 codebook 的码，再由声码器还原成波形。

为什么需要它：单个 codebook 表达力不够（音质差）；用多个 codebook 残差叠加，才能在低比特率下保留细节。代价是模型每步要处理“一摞”码而不是一个 token。

本 PR 里怎么用：`cb0_per_row` 取的就是每个请求“codebook-0”的码，它被用来当作下一步自回归的输入接力（input_ids）。代码里的 `_cg_codes_BN` 形状是 `[Batch, Num_codebooks]`，`[:, 0]` 就是取 0 号码本那一列。

► Model Runner（模型运行器）与多阶段流水线

它是什么：在 SGLang 体系里，**Model Runner** 是真正“把一个 batch 喂进模型、跑一次前向、收集输出”的执行单元。SGLang-Omni 把 TTS/Omni 拆成 `preprocessing → AR engine → codec → vocoder` 等多个阶段（stage），每个阶段可以跑在不同进程甚至不同 GPU 上。

为什么需要它：把“取数据 / 前向 / 采样 / 收集结果”封装在 runner 里，调度器（scheduler）只管“谁先跑、跑多大 batch”，职责清晰；多阶段拆分则让 CPU 预处理、GPU 推理、声码器解码能各自并行、互不阻塞。

本 PR 里怎么用：改的就是 `HiggsTTSTModelRunner` 里“收集这一步输出”的逻辑。`post_decode`（同步）、`post_decode_launch` / `post_decode_resolve`（异步两半）都复用同一个 `_decode_collect_host`，所以加一个参数就能让两条路径各取所需。

► 异步解码 & Host→Device 拷贝 / 隐式同步

它是什么：GPU 计算是异步排队的——CPU 把任务塞进 CUDA stream 就返回了，GPU 在后台慢慢算。"异步解码"利用这一点：GPU 先把下一步算起来（launch），CPU 在后面一步把上一步结果搬回主机（resolve）。Host→Device（H2D） / Device→Host（D2H）拷贝就是 CPU 内存与显存之间搬数据。

为什么需要它：如果每一步都"算完→立刻把结果拷回 CPU→等拷完→再算下一步"，GPU 会频繁空等（这种等待叫**隐式同步**，比如 `.item()`、跨设备建张量都可能触发）。异步流水线把搬运藏到下一步计算的背后，GPU 几乎不停。

本 PR 里怎么用：旧代码在异步 resolve 里 `device=GPU` 地建张量，相当于在"本该轻量的整理步"又戳了一下 GPU、多搬一趟数据。改成在 CPU 上建，既匹配它"马上要 `.tolist()`"的用途，又避免打断 GPU 抢跑。Benchmark 显示吞吐 +13~15%、RTF 降约 6%，正是这种"去掉一次小同步"在高并发下被放大的效果。

► CUDA Graph 与 one-step lookahead（抢跑一步）

它是什么：CUDA Graph 把一串固定的 GPU kernel"录制"成一张图，之后整张图一次性重放，省掉每个 kernel 的启动开销（launch overhead）。代码里 `_cg_*` 前缀（cg = CUDA Graph）的缓冲区就是为重放预分配的固定内存。

为什么需要它：TTS 解码每步很小、kernel 很多，CPU 逐个发射 kernel 的开销会成为瓶颈。录成图重放后，CPU 只需"按一次播放键"。配合 lookahead（提前一步发起下一步），GPU 利用率进一步拉满。

本 PR 里怎么用：`post_decode_launch` 在重放图后**不做 host 同步**就直接发布 GPU 上的 cb0，让自回归链条不断；真正的主机整理滞后一步在 `resolve` 完成。本 PR 保证了这个滞后整理只产出 CPU 张量，不再回头碰 GPU。

性能数据（PR 自带 · Seed-TTS EN，并发 8，async decode 开启）

指标	Main (旧)	本 PR	变化
吞吐 Throughput (req/s)	8.400	9.645	+14.8%
输出 token 吞吐 (tok/s)	986.3	1116.6	+13.2%
音频吞吐 (s/s)	37.10	41.97	+13.1%
延迟均值 (s)	0.891	0.828	-7.1%
RTF 均值（越低越好）	0.2073	0.1940	-6.4%
WER 语料级（音准）	1.16%	1.11%	≈ 持平

5. 我能学到什么 / 延伸思考

- "为正确性付出的代价应当只付一次"。同一份整理逻辑被同步/异步两条路复用，作者没有复制粘贴，而是用一个参数把"设备归属"这件唯一的差异抽出来——既复用又不牺牲性能。
- 性能优化常常藏在"无害但多余"的细节里。这里没有改算法、没有上新 kernel，只是删掉一次"GPU 上建张量再拷回"的来回，就能在高并发下放大成两位数吞吐提升。
- 区分"驱动计算的张量"和"给用户看的张量"。前者必须在 GPU、参与下一步；后者只是结果快照，放 CPU 最省。识别清楚数据的"用途"，才知道它该住在哪。
- 延伸前置知识：CUDA stream / event 与异步执行模型、`torch` 的 pinned memory 与 `non_blocking=True` 拷贝、RVQ 神经音频编解码器、SGLang 的 scheduler↔model-runner 分工。

PR #691

注意力后端

LLaDA2-Uni

多模态

[LLaDA2-Uni] Set default attention backend to flashinfer

作者 @btw616 · 状态 merged · [查看 PR](#) ↗

一句话：给多模态扩散语言模型 LLaDA2-Uni 显式钉死注意力后端为 **flashinfer**，修复“关掉 CUDA Graph 后注意力后端被自动选成不支持的那个”的连带 bug。

2. 它解决了什么问题

LLaDA2-Uni 是一个 多模态扩散语言模型 (DLLM)，能吃文本+图像、产出文本。它唯一被完整验证支持的注意力后端是 FlashInfer。但之前代码从没显式声明过这件事——它是“顺带”被保证的：早先默认开启 CUDA Graph，而开 CUDA Graph 时框架恰好会把后端选成 FlashInfer。换句话说，正确的后端是靠一个无关设置的副作用蒙对的。

最近有个改动默认关掉了 LLaDA2-Uni 的 CUDA Graph（这条实验性 DLLM 路径开图不稳定）。这一关，“顺带保证后端”的隐含链条就断了：框架的自动后端选择逻辑可能挑中一个 LLaDA2-Uni 根本不支持的后端，于是启动/推理直接出问题。本 PR 把这层隐式依赖变成显式声明。

类比：你家的灯一直亮着，是因为“开空调时顺手也把灯打开了”。某天有人说“空调太吵，默认别开了”，结果灯也黑了——因为灯从来没有自己的开关，一直蹭空调的。这个 PR 就是给灯装上专属开关，不再依赖空调。

3. 具体做了什么改动

仅一处，文件 **sglang_omni/models/llada2_uni/stages.py**。在构造 thinker 阶段的 server 参数时，把 overrides 字典从“只关 CUDA Graph”扩展为“显式指定 flashinfer + 关 CUDA Graph”：

```
- overrides = {"disable_cuda_graph": True}

+ overrides: dict[str, Any] = {
+     "attention_backend": "flashinfer",
+     "disable_cuda_graph": True,
+ }

overrides.update(server_args_overrides or {})
```

逻辑前后对比：

- 改动前：只设 **disable_cuda_graph=True**。 **attention_backend** 留空 → 交给框架自动推断 → 在“无 CUDA Graph”语境下可能选到非 FlashInfer 后端 → 崩。

- 改动后：显式 `attention_backend="flashinfer"`，不再让框架猜。同时保留 `overrides.update(...)`，意味着用户若在配置里显式传了别的后端，仍可覆盖——默认值安全，但不剥夺高级用户的控制权。

设计细节：注意 `overrides.update(server_args_overrides or {})` 放在最后一行执行，所以这里写的是默认值而非强制值——这是“安全默认 + 可覆盖”（sane default, still overridable）的经典写法。

4. 涉及的知识点（重点·小白展开）

► Attention Backend（注意力后端）是什么

它是什么：Transformer 的核心计算是“注意力”（attention）。同一个数学公式，可以由不同的高性能实现来算——FlashInfer、FlashAttention、Triton、纯 PyTorch 等，这些可互换的实现就叫“注意力后端”。它们各自对硬件、数据布局、KV-cache 排布、是否支持变长/分页有不同要求。

为什么需要它：注意力是推理里最吃显存带宽和算力的部分。挑对后端能显著提速、省显存；但不是每个后端都支持每种模型/特性（比如某些采样模式、分页 KV、特定 mask）。所以“模型 ↔ 后端”是有兼容性约束的。

本 PR 里怎么用：LLaDA2-Uni 这条实验路径只在 FlashInfer 上被验证过。PR 把它钉死为默认后端，避免框架自动选择时挑到不兼容的实现。

► FlashInfer 简介

它是什么：FlashInfer 是一个面向 LLM 推理服务的高性能注意力/采样算子库，提供分页 KV-cache、各种 prefill/decode 注意力核、以及对 SGLang 这类引擎友好的接口。SGLang 把它作为重点支持的后端之一。

为什么需要它：它针对“服务场景”（变长序列、连续批处理、分页 KV）做了深度优化，比通用实现更适合高并发推理。

本 PR 里怎么用：就是被显式指定的那个值 `"flashinfer"`。文档配套也补了一句：“thinker 在这条实验 DLLM 路径下默认关闭 CUDA graph”。

► CUDA Graph 为什么会“顺带”决定后端

它是什么：前面讲过，CUDA Graph 把固定 kernel 序列录制后整体重放。但要能录制，所有 kernel 形状必须固定、且后端要支持“图捕获（graph capture）”模式。框架在“决定开图”时，往往会连带把注意力后端切到支持图捕获的那个（即 FlashInfer）。

为什么需要它：这正是隐式耦合的来源——两个本应独立的配置（是否开图、用哪个后端）被框架的自动逻辑绑定在了一起。一旦改了其中一个，另一个的取值就跟着变。

本 PR 里怎么用：作者识别出这条隐式链，主动解耦：无论开不开图，后端都明确是 FlashInfer。这就是“消除隐式依赖、改为显式契约”的工程思想。

► 扩散语言模型 DLLM & thinker 阶段

它是什么：主流 LLM 是自回归（一个一个 token 往外吐）；**扩散语言模型（Diffusion LLM）** 则像图像扩散那样，从“全是 mask 的占位”出发，**分多轮并行去噪/填空**，逐步把整段文本确定下来。LLaDA2 系列就属于这类。“thinker”是 Omni 流水线里负责“理解+生成文本表征”的核心阶段。

为什么需要它：扩散式生成在某些场景能更好地做全局规划、并行填充，是当前的研究热点。但它的调度/缓存模式和自回归不同，对引擎适配提出新要求（见下一张卡片 #670）。

本 PR 里怎么用：改的就是 `create_sglang_dllm_thinker_executor_from_config`——给 DLLM thinker 阶段配置后端。它和 #670 修的是同一个模型，一个修后端、一个修调度器，互为补充。

5. 我能学到什么 / 延伸思考

- “靠副作用蒙对”是技术债。当一个正确行为依赖于另一个无关配置的副作用时，它随时会因为那个配置的变动而无声崩坏。把隐式依赖写成显式声明，是消除这类脆弱性的标准手法。
- 安全默认 + 仍可覆盖。把默认值放进 overrides 字典、再让用户配置 `update` 在其后，是兼顾“开箱即用”和“高级可控”的小而美的写法。
- 一行代码的价值取决于它背后的因果链。这个 PR 只加了一行，但价值在于作者讲清了“关图 → 自动选后端 → 选错 → 崩”的完整链路。读 PR 要读因果，不要只读 diff。
- 延伸前置知识：FlashInfer/FlashAttention 的差异、注意力后端与 KV-cache 布局的兼容关系、扩散语言模型的采样过程、SGLang server_args 的后端自动选择逻辑。

PR #670

调度器适配

文档

LLaDA2-Uni

Verify SGLang-Omni cookbook docs (含一处 DLLM 调度器实质修复)

作者 @MelodyyyYin · 状态 merged · 查看 PR ↗

一句话：表面是“核对/补全各模型 cookbook 文档”，但夹带了一处关键代码修复——让 LLaDA2-Uni 的扩散调度器适配当前 SGLang 已经改过的 `PrefillAdder` API，使 `/v1/chat/completions` 文本输出真正跑通。

2. 它解决了什么问题

SGLang-Omni 把 `sglang` 作为底层推理引擎依赖。但开发镜像里跑的是 `sglang 0.0.0.dev1`（比锁定的 `0.5.8` 更新），上游把 `PrefillAdder` 的接口改了：

- 它的 `__init__` 不再接受 `dllm_config` 参数；
- 原来的辅助属性 `PrefillAdder.dllm_staging_reqs` 被删除了。

于是 LLaDA2-Uni 旧的扩散调度器路径在“还没碰到模型”时就抛异常——也就是说，`/v1/chat/completions` 对这个模型根本调不通。本 PR 把调度器改写到新 API 上，让文本对话请求能正常返回（已在 dev 镜像上实测：返回连贯文本、`finish_reason: stop`、HTTP 200）。

上游漂移 (upstream drift) 的典型示例：你依赖的库悄悄改了内部 API，你的适配层就断了。这类问题不在你的代码里、却让你的功能挂掉，排查尤其费神。

3. 具体做了什么改动（核心：`scheduling/dllm_scheduler.py`）

(a) chunked-prefill 尺寸来源更稳健

```
+ self.chunked_prefill_size = (  
+     getattr(dllm_config, "block_size", None) or server_args.chunked_prefill_size  
+ )
```

扩散模型按“块 (block)”去噪，块大小更贴合它的语义；拿不到就回退到 server 通用值。

`getattr(..., None) or ...` 是 Python 里“优先取 A，没有就取 B”的惯用写法。

(b) 构造 PrefillAdder 时不再传被删掉的参数

```
adder = PrefillAdder(

    ..., self.server_args.max_prefill_tokens,

    - self.server_args.chunked_prefill_size,

    - dlm_config=self.dlm_config,      # 新版已不接受这个参数

    + self._chunked_prefill_size,

    + prefill_max_requests=1,          # 显式限制：本阶段一次只跑 1 个请求

)
```

(c) 用 `can_run_list` 取代被删的 `dlm_staging_reqs`

```
- for req in adder.dlm_staging_reqs:    # 旧 helper，已不存在

+ for req in adder.can_run_list:        # 用现有的"可运行列表"自己维护暂存队列

    if req.rid not in staging_rids:

        self._staging_queue.append(req)
```

扩散请求要反复调度多轮才结束，所以调度器需要把"正在进行"的请求留在自己的 staging 队列里；既然上游不再帮忙提供这个列表，就用现有的 `can_run_list` 自己维护。

(d) 结果回填：兼容"单请求扁平 token 列表"

```
# 本阶段一次只跑 1 个请求，模型可能返回"扁平的一串 token id"

# 而不是"每请求一份"的嵌套结构，这里做归一化：

+ if len(batch.reqs) == 1 and (not token_ids or isinstance(token_ids[0], int)):

+     token_ids_per_req = [token_ids]    # 包一层，统一成 [[t0,t1,...]]

+ else:

+     token_ids_per_req = token_ids

+ for req, req_token_ids in zip(batch.reqs, token_ids_per_req):

+     for token_id in req_token_ids:

+         req.output_ids.append(int(token_id))

+         req.check_finished()

+         if req.finished():

+             break
```

作者还贴心地留了一条护栏注释：如果将来把 `prefill_max_requests` 提到 1 以上，这段"扁平列表"分支必须连同调度上限一起重审，否则 `zip()` 会把每个请求错配成单个 `int`。这是"为未来的自己/同事埋好警示牌"的好习惯。

文档侧改了什么（同 PR 一并合并）

- 新增 cookbook：Fish Audio S2-Pro、Qwen3-ASR、Whisper ASR（后者标注"实验性、未做正确性验证，优先用 Qwen3-ASR"）。
- 纠正 Higgs 公共语音字段：去掉文档里并不存在的 `reference_codes` / `reference_text`，改为真实暴露的 `references` / `ref_audio` / `ref_text`。
- 给 Qwen3-Omni 补 `max_completion_tokens`（OpenAI 兼容别名）。
- 首页特性表加入 `/v1/audio/transcriptions` 端点与 ASR 模型，明确 LLaDA2-Uni 当前"只支持文本输出，图像/交错生成尚未接通"。

4. 涉及的知识 (重点·小白展开)

► Scheduler (调度器) 与 PrefillAdder

它是什么：调度器是推理引擎的"交通指挥"：决定当前这一拍把哪些请求、组成多大 batch 喂给 GPU。

PrefillAdder 是 SGLang 里负责"prefill 阶段往 batch 里塞请求"的组件——它会按显存预算、token 上限、KV-cache 容量来决定能塞几个、塞多少。

为什么需要它：推理是多请求共享一块 GPU。塞太多会 OOM，塞太少浪费算力。PrefillAdder 就是把"能塞的请求"精确算出来的那段逻辑。

本 PR 里怎么用：扩散调度器复用 PrefillAdder 来组 batch；上游改了它的签名（去掉 `dllm_config`、删了 `dllm_staging_reqs`），本 PR 把调用方式迁移到新接口，并用 `prefill_max_requests=1` 显式表达"这条实验路径一次只跑一个请求"。

► Prefill vs Decode / Chunked Prefill

它是什么：自回归推理分两段：**Prefill**（一次性并行处理整段输入 prompt，建好 KV-cache）和 **Decode**（之后一个一个往外吐 token）。**Chunked Prefill**：当 prompt 很长时，把 prefill 切成若干小块分批做，避免一次占满显存、也让长 prompt 不至于饿死其他请求。

为什么需要它：长输入的 prefill 是显存/算力尖峰；分块能平滑这个尖峰，提升整体并发与公平性。

本 PR 里怎么用：(a) 处优先用扩散模型自己的 `block_size` 作为 chunk 尺寸——因为 DLLM 本来就是按块去噪的，块大小语义最贴合。

► 扩散调度的特殊性：为什么要"反复调度同一请求"

它是什么：自回归请求每步产生一个 token、走很多步直到 EOS；扩散请求则是对**整个序列**做多轮去噪，每一轮都要再进行一次模型，直到所有位置都"定稿"。

为什么需要它：因为同一个请求要被调度器拉起好多轮，调度器必须自己记住"哪些请求还在去噪中"（staging 队列），不能 prefill 一次就丢掉。

本 PR 里怎么用：(c) 处用 `can_run_list` 重建这个 staging 队列，正是为了在新 API 下继续维持"扩散请求的多轮生命周期"。

5. 我能学到什么 / 延伸思考

- 适配层 (adapter) 是依赖漂移的缓冲垫。当你把第三方引擎包进自己的抽象里，对方升级就可能击穿你的假设。把"对外部 API 的调用"集中、并用 `getattr(..., default)` 等防御写法，能降低被上游改动击穿的概率。
- 给未来留护栏注释。作者在"扁平列表"分支显式写明"若 `prefill_max_requests > 1` 必须重审"，把一个隐藏假设变成可见警告——这是高质量工程代码的标志。
- 文档 PR 也可能"夹带"实质修复。读 PR 别只看标题；这条标题是"verify docs"，真正的金子是调度器迁移。审阅时务必看 diff 全貌。

- 延伸前置知识：prefill/decode 两阶段、KV-cache 与显存预算、连续批处理（continuous batching）、扩散语言模型的去噪调度、语义化版本与依赖锁定（dependency pinning）。

Add support for pyAV-based mp3, aac, opus audio encoding

作者 @haojin2 · 状态 merged · 查看 PR ↗

一句话：把 TTS 输出的 mp3/aac/opus 编码从依赖外部 `ffmpeg` 命令行的 `pydub` 切换到进程内的 PyAV，并自动处理各编码器对采样率的限制（必要时线性重采样），更稳、更可控、可降级回退。

2. 它解决了什么问题

TTS 模型直接产出的是 24kHz 的浮点波形（PCM）。但用户通过 `/v1/audio/speech` 拿到的常常想要 mp3/aac/opus 这种压缩格式。旧实现用 `pydub` 来压缩，而 `pydub` 本质是去 shell 调用系统里的 `ffmpeg` 二进制——这带来两个麻烦：

- 环境脆弱：容器里不一定装了对的 `ffmpeg`、或版本不匹配，编码就失败。
- 采样率坑：不同编码器对采样率有硬性限制（比如 Opus 只接受 8/12/16/24/48kHz）。直接把 24k 之外的采样率喂进去会报错或产出坏文件。

本 PR 改用 PyAV（FFmpeg 的 Python 原生绑定，进程内调用，不依赖外部命令行），并为每种格式预置“合法采样率白名单”，遇到非法采样率就先线性重采样到最近的合法值，再编码。万一 PyAV 也失败，还会优雅降级回 `pydub`，再不行回退 WAV——三层兜底。

3. 具体做了什么改动（核心：`sglang_omni/client/audio.py`）

(a) 预置各编码器的“合法采样率”配置表

```
PYAV_ENCODE_CONFIGS = {
    "opus": {"container": "ogg", "codecs": ("libopus", "opus"),
            "valid_rates": {8000, 12000, 16000, 24000, 48000}},
    "aac": {"container": "adts", "codecs": ("aac",),
            "valid_rates": {8000, 16000, 22050, 24000, 44100, 48000, ...}},
    "mp3": {"container": "mp3", "codecs": ("libmp3lame", "mp3"),
            "valid_rates": {8000, 16000, 22050, 24000, 44100, 48000, ...}},
}
```

每种格式记录三件事：用哪个容器（封装格式）、优先尝试哪些编码器（codec，给出备选名以适配不同 FFmpeg 构建）、以及该编码器能接受的合法采样率集合。

(b) 非法采样率 → 线性重采样到最近合法值

```
def _encode_with_pyav(audio, sample_rate, container_format, codecs, valid_rates):
    if sample_rate not in valid_rates:
        nearest_rate = min(valid_rates, key=lambda r: abs(r - sample_rate))
        audio = _resample_linear(audio, sample_rate, nearest_rate) # 重采样
        sample_rate = nearest_rate

    container = av.open(buf, mode="w", format=container_format)
    stream = None
    for codec in codecs:      # 逐个尝试候选 codec，挑第一个本机支持的
        try:
            stream = container.add_stream(codec, rate=sample_rate); break
        except av.CodecError:
            continue
    ...
    # FFmpeg 这些 codec 要 float-planar (flt) 格式、单声道
    frame = av.AudioFrame.from_ndarray(audio.reshape(1, -1), format="flt", layout="mono")
    for packet in stream.encode(frame): container.mux(packet)
    for packet in stream.encode(None): container.mux(packet) # flush 收尾
```

这里有两个值得注意的工程点：① codec 候选回退——先试 `libopus`，不行再试内建 `opus`，因为不同 FFmpeg 编译版本带的编码器名字不同；② `stream.encode(None)`——音频编码器内部有缓冲，传 `None` 是“冲刷 (flush)”信号，把最后残留的几帧也吐出来，否则文件尾部会丢音。

(c) 重采样实现：用 numpy 线性插值，零额外依赖

```
def _resample_linear(audio, orig_sr, target_sr):
    if orig_sr == target_sr: return audio      # 相等直接返回
    duration = audio.shape[0] / float(orig_sr)
    new_len = max(int(round(duration * target_sr)), 1) # 按时长换算新样本数
    new_idx = np.linspace(0.0, audio.shape[0] - 1, num=new_len)
    return np.interp(new_idx, old_idx, audio).astype(np.float32) # 线性插值
```

这是最朴素的重采样：把新采样点位置映射回原信号、做线性插值。音质不如多相滤波，但无需 `scipy/librosa`、零依赖、足够应付“凑到合法采样率”这种小幅调整。

(d) 三层降级的调度逻辑

```
+ if fmt in ("opus", "aac", "mp3"):

+     try:

+         return _encode_with_pyav(arr, sample_rate, ...config...), mime # ① 首选 PyAV

+     except Exception as e:

+         logger.warning("PyAV %s encoding failed (%s); falling back to pydub/WAV", ...)

    try:

        from pydub import AudioSegment # ② 退回 pydub

        ...

    except ...:

        return encode_wav(arr, sample_rate), ... # ③ 最后保底 WAV
```

FLAC 仍走 `soundfile` 单独分支。整体策略：能用更好的就用更好的，但永远有兜底，绝不让用户拿到一个空响应。

新增单测：`tests/unit_test/client/test_audio.py`

为 opus/aac/mp3 各加了"正常采样率"和"非法采样率（触发重采样）"两组用例，并用文件魔数校验编码正确：Ogg 文件以 `b"OggS"` 开头、MP3 以 `b"ID3"` 开头、AAC(ADTS) 帧以 `0xFF 0xFx` 开头。这种"按字节签名断言格式"是音频/二进制测试的经典手法。

4. 涉及的知识点（重点·小白展开）

► PCM / 波形 / 容器 / 编解码器（codec）四件套

它是什么：PCM/波形是声音的"原始数字采样"（一串浮点数）；**编解码器 codec**（mp3/aac/opus...）是把波形压缩成更小字节流的算法；**容器**（ogg/adts/mp3 文件）是把压缩后的数据流装进一个带头部、能被播放器识别的"盒子"。

为什么需要它：裸 PCM 体积大、不带元信息；压缩后小很多，但必须装进容器、声明采样率/声道，播放器才认。

本 PR 里怎么用：配置表里 `container`（盒子）和 `codecs`（压缩算法）是分开指定的——比如 opus 用 `ogg` 容器装 `libopus` 编码的数据。

► PyAV vs pydub：进程内绑定 vs 外部命令

它是什么：pydub 是高层封装，运行时去 shell 调系统的 ffmpeg 可执行文件；PyAV 是 FFmpeg 库（libav*）的 Python 原生绑定，在进程内直接调 C 库，不依赖外部命令、不开子进程。

为什么需要它：外部命令依赖环境装没装、版本对不对、PATH 找不找得到，部署脆弱且报错难定位；进程内绑定可控、可捕获异常、可精确控制采样率/帧格式。

本 PR 里怎么用：把首选编码路径换成 PyAV，同时保留 pydub 作为降级——既拿到稳定性，又不破坏老环境的兼容。

► 采样率 & 重采样（resampling）

它是什么：采样率是每秒采多少个声音样本（24000Hz = 每秒 2.4 万点）。重采样是把一段音频从一个采样率换算到另一个（点数随之改变）。

为什么需要它：每种压缩编码器只支持特定采样率集合。模型输出 24kHz 多半 OK，但若上游给了 44.1k/12345 这种值，就得先换到编码器认的最近值。

本 PR 里怎么用：`_resample_linear` 用线性插值做轻量换算；`valid_rates` 白名单 + `min(..., key=abs(r-sr))` 找最近合法采样率。

5. 我能学到什么 / 延伸思考

- "优雅降级"是服务化代码的底线。三层兜底（PyAV → pydub → WAV）保证用户永远拿得到能播的音频，哪怕画质打折也不返回错误——这是面向最终用户接口的健壮性思维。
- 把外部进程依赖换成进程内库，是提升可部署性的常见升级。少一个"系统得装某个二进制"的前提，容器就更省心、错误更可捕获。
- 用魔数测二进制格式。不解码、只看文件头几个字节，就能稳定地断言"编码确实成功且格式正确"，比"能不能播放"更适合自动化测试。
- 延伸前置知识：数字音频基础（采样率/位深/声道）、FFmpeg 的容器与编码器关系、planar vs interleaved 音频帧格式、编码器内部缓冲与 flush、Nyquist 采样定理与重采样滤波。

PR #647

基准 / CI

Qwen3-ASR

并发扫描

[CI] Align Qwen3-ASR stage-1 gate to concurrency=2 and add ASR concurrency-scaling benchmark

作者 @MelodyyyYin · 状态 merged · [查看 PR](#) ↗

一句话：CI 的 ASR 关卡实际在并发=2 下跑，但速度基线却还是并发=32 的宽松旧值（2~5 倍太松，挡不住性能退化）；本 PR 把基线对齐到真实并发，并新增一个可复现的 ASR 并发扫描基准脚本。

2. 它解决了什么问题

CI（持续集成）里有个“门禁测试”（gate）：每次提交都跑一遍 Qwen3-ASR 转写 20 条样本，检查 WER（词错率）和速度是否达标。问题是——这个 gate 的速度阈值用的还是早先并发=32 测出来的数字（吞吐 10.3、延迟均值 0.193...），可 gate 实际是按 并发=2 在跑的。

并发=2 时真实性能好得多（吞吐 16.3、延迟 0.121...），于是旧阈值比实际宽松了 2~5 倍——这意味着即使引擎性能退化了，gate 也照样放行，形同虚设。本 PR 把阈值重新校准到“gate 真正运行的那个工作负载（并发=2）”上。

类比：体检设定“100 米跑进 20 秒算合格”，但其实你每次只走 50 米。哪怕你状态明显变差，也永远在 20 秒内“达标”。这个 PR 把标准改成“按你真正跑的 50 米来定”，体检才有意义。

一个有意思的洞见（PR 描述给出）：在 20 条这种小负载上，并发=2 反而比并发=32 快。因为样本太少时，高并发主要在测“突发排队/尾延迟”，而非稳态吞吐；ASR 的高扇出（fan-out）只有在大负载上才划算。WER 则基本与并发无关。

3. 具体做了什么改动

- `tests/test_model/test_qwen3_asr_ci.py`：把 stage-1 速度阈值校准到并发=2 的实测值；把代码内默认并发从 32 改为 2（让本地运行、阈值调优工具和 CI 跑的是同一个工作负载）。WER 阈值不变（与并发无关）。同时把原本散落在测试里的“转写+打分”逻辑抽取并复用新基准脚本里的函数。
- `benchmarks/eval/benchmark_qwen3_asr_concurrency.py`（新增 494 行）：一个可复现的 ASR 并发扫描器，直接对运行中的 Qwen3-ASR 路由转写 SeedTTS 参考音频，按并发档位（1,2,4,...,64）多次重复，报告语料/逐条 WER、墙钟时间、吞吐、延迟分位、RTF、以及每个 worker 的路由均衡度。
- 配置/文档：修正 `tune-ci-thresholds` 配置里“gate 跑在 32”的过时注释；在 `benchmarks/README` 记录新脚本与“并发-扩展”结论。

设计亮点：gate 与 benchmark "同源"

最漂亮的一点是去重：测试文件不再自己写一套转写/打分代码，而是直接 `import` 基准脚本里的 `run_asr_transcription` 和 `build_asr_eval_results`。这样"CI 门禁"本质上就等于"跑一遍基准 + 套阈值"，两者永远用同一套度量口径，不会出现"测试和基准算法漂移"的问题。

```
- # 测试里自己写一套 _transcribe_with_omni / _percentile / 手算 WER... (删掉~150 行)

+ from benchmarks.eval.benchmark_qwen3_asr_concurrency import (
+     build_asr_eval_results, run_asr_transcription, # 与 benchmark 共享同一条路径
+ )

+ outputs, wall_clock_s = asyncio.run(run_asr_transcription(seedtts_en_samples, ...))

+ results = build_asr_eval_results(seedtts_en_samples, outputs, wall_clock_s, "en", ...)
```

还有个埋在 `send` 函数里的实战注释："不要发 temperature=0——Qwen3-ASR 在纯贪心解码下会退化，服务端会把它顶到 0.01"。这种"魔鬼细节"正是基准脚本沉淀经验的价值。

4. 涉及的知识点（重点·小白展开）

► 并发 / 扇出（concurrency, fan-out）与连续批处理

它是什么：并发指同时在途的请求数。推理引擎用连续批处理（continuous batching）把多个在途请求动态拼成一个 batch 喂 GPU——谁完成谁退出、空位立刻补新请求，GPU 始终满载。扇出就是客户端一次性放出多少并发请求。

为什么需要它：高并发能把 GPU 喂饱、提升稳态吞吐；但在小负载上，高并发主要制造排队和尾延迟，并不提速。所以"最优并发"和工作负载大小强相关。

本 PR 里怎么用：新脚本就是用来"扫描"不同并发档位、量化它们对吞吐/延迟/WER 的影响，从而为"小门禁用并发 2、大评测用并发 32"提供数据支撑。

► WER / RTF / 延迟分位 (p95/p99) 这些指标怎么读

它是什么：WER (Word Error Rate, 词错率) 衡量转写质量, 越低越好; RTF (Real-Time Factor, 实时率) = 处理耗时 / 音频时长, <1 表示比实时还快; p95/p99 延迟是"95%/99% 的请求都比这个值快", 反映尾部体验。

为什么需要它：只看均值会掩盖长尾。p95/p99 才暴露"最慢的那一小撮"; RTF 让不同时长音频可比; WER 衡量"快了但准不准"。

本 PR 里怎么用：基准脚本对每个并发档位都报全套指标, 并跨多次重复取 mean/min/max, 给出可复现、抗噪声的"并发-扩展曲线"。

► CI 门禁 & 阈值校准 (threshold calibration)

它是什么：CI 门禁是合并前自动跑的"红线测试", 不达标就拦下。阈值校准是把这些红线设到"刚好能放过正常波动、又能挡住真实退化"的位置。

为什么需要它：阈值太松 = 退化漏网; 太紧 = 正常抖动也误报。关键是阈值必须基于"门禁真正运行的那个工作负载"来定——这正是本 PR 修的 bug。

本 PR 里怎么用：把阈值从并发=32 的旧值改到并发=2 的实测值, 并让默认并发、本地运行、tune-ci-thresholds 工具三者对齐到同一负载。

5. 我能学到什么 / 延伸思考

- "测试要测你真正在跑的东西"。当门禁的阈值和它实际运行的负载脱节, 门禁就退化成橡皮图章。校准必须基于真实工作负载。
- 让测试与基准同源, 消灭"度量漂移"。把转写/打分逻辑抽成共享函数, CI = 跑基准 + 套阈值, 从根上避免"两套算法各算各的"。
- 性能直觉要用数据验证, 而非想当然。"并发越高越快"在小负载上是错的; 本 PR 用扫描数据证明了这点, 并据此分场景选并发。
- 延伸前置知识: continuous batching 与 PagedAttention、吞吐 vs 延迟的权衡、长尾分位统计、ASR 评测流程 (归一化 + jiwer 计算 WER)、负载测试方法论。

其余 9 个文档 / 基建类 PR（无核心逻辑改动）

这些 PR 不改推理逻辑，但反映了项目当日的运营重心：Higgs Audio 文档大刷新 + 正式 Docker 发布。

PR	标题	作者	做了什么
#694	[Doc] Revise supported languages	@SilinMeng0510	支持语言一节改为给链接、删掉冗长列表。
#690	[Docker] Release Formal Docker	@zhaochenyang20	发布正式版 Docker 镜像（基建里程碑：从 dev 镜像走向正式分发）。
#688	[Docs] higgs-tts 语言分级计数订正	@SilinMeng0510	修正各 WER/CER 档位语言数（82→83 / 18→17，总数仍 100）。
#685	[Docs] Update Higgs Audio Cookbook	@zhaochenyang20	更新 Higgs Audio cookbook。
#684	[Docs] Refresh Higgs TTS cookbook	@SilinMeng0510	对齐官方 model card、补实测吞吐数、精简高亮块。
#683	[Docs] Quick Fix Higgs	@zhaochenyang20	修 lint。
#681	[Refresh] Higgs Performance	@zhaochenyang20	刷新 Higgs 性能数据。
#673	[Docs] Higgs TTS cookbook : voices / control-token / step-599 benchmark	@SilinMeng0510	重做语音示例、内联控制 token 使用规则、刷新到最新 checkpoint 评测。
#652	[DOC] benchmark/README 新增 flag 说明	@haojin2	给基准脚本新参数补文档。

当日观察：9 个文档 PR 里有 6 个围绕 [Higgs Audio/TTS](#)，叠加 #692 的 Higgs 性能优化与 #690 的正式 Docker——可见团队正在为 [Higgs Audio](#) 的一次正式对外发布做收尾（文档对齐 model card + 性能刷新 + 正式镜像）。同时 ASR（Qwen3-ASR/Whisper）正作为新模态被逐步接入并文档化。